

Chips Operational Semantics

Anna <3

May 27, 2026

Introduction

Chips permits its users to describe models of systems executing programs. Therefore, it does not only allow to design algorithms, but also cares about notions such as “program embedding”, “dataflows” or “device interconnections”. The simultaneous need to describe both algorithms and devices implies a syntax that mixes elements of both imperative and declarative (synchronous) paradigms. Additionally, the coordination of many components in a single system requires a new kind of (collective) primitives that allows the specification of the data evolution when spread or collected among crowds of devices, where local information matters. This document describes a formal interpretation for the models the Chips language allows to implement. Thank to this description, it becomes possible to draw proofs or formal property definitions for any given Chips model at a system-wide abstraction level. Section 1 explores briefly the main system description concepts of the language. Then, Section 2 precises formally such concepts to demonstrate its ability to implement the concept of Hierarchical Control Motifs for complex systems design.

1 Overview of Chips modeling concepts

Chips is a language for defining Cyber-Physical Systems (CPS) putting the emphasis on the compositional aspect of each functionalities. The broad definition of “systems” that we adopt can be summed up as “a set of *components* in interaction that together *provide a service*”. In Chips, the notion of providing a service leads to the concept of *function*, while the notion of components is embodied by the concept of *node* embedding such functions. To permit the coordination of functions accross nodes, Chips introduces the concept of user defined *collective primitives*. These three ideas are detailed in the following subsections.

1.1 Functions

Functions are the abstraction chosen to represent running processes in a Chips model. When defining them, one must specify:

```

1 logical accumulatedSumProduct(int x) init {
2     int sum = 0;
3     int prod = 1;
4 } then {
5     sum = sum + x;
6     prod = prod * x;
7 }->accSum(sum)
8 ->accProd(prod)

```

Figure 1: Definition of a Chips function

- a set of typed input variables and their potential default values,
- an initializing process that defines inner state variables and their potential default values,
- an updating process that defines the evolution of the inner state variables according to the values of input variables, and
- a set of named and type-deduced output expressions.

To showcase the syntax of Chips applied to the definition of a function, Figure 1 presents the definition of a function that computes simultaneously the sum and the product of its input.

Broadly speaking, when instantiated, a function applies its initializing protocol (the *init* section). Then, it applies its updating process (the *then* section) everytime all its input variables are updated exactly once. If an input variable has a default value, Chips considers it has already been updated once at the initialization of the model. Therefore, when connecting input and outputs of functions, one must care about the presence/absence of default values for inputs of components to correctly understand the way dataflow evolve or why some compositions may lead to delayed system deadlock, as shown in figure 2.

From the model composition perspective, they can be seen as black boxes providing an interface being their set of inputs and their set of outputs. A functional block with all its inputs having a default value is able to *trigger* the components depending on it once because it has all the required information to compute its *then* section once. Remark that functional blocks with no input parameter are always able to compute the content of their *then* section because such computation does not depend on pre-computed data. So they never can be a source of deadlock for the system.

The composition of functional blocks leads to the creation of a directed graph, which we will call the *functional graph*. The vertices are the functions and the edges are the input/output linkage between the functions. The presence of loops in the graph signifies that at least a part of the designed system retro-acts on itself. Just as the wheels of a vapor train, all functions run their *then* sections altogether, one function giving the pace for the next ones. Therefore,

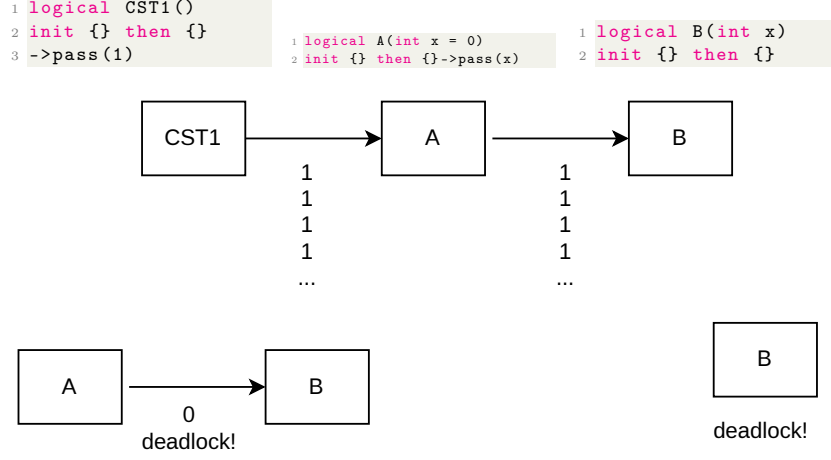


Figure 2: Presence or absence of deadlocks for sets of functions defined with or without default values for their input.

no clock is explicitly defined for the systems. The execution of the *then* section of any component can serve as an implicit clock tick.

The graph formalism applied to the functional dependency of the model elements allow to perform early checking of properties of the model before it is instantiated on a real system.

1.2 Nodes

Nodes are an abstraction of the Chips models that represent locations. If two events happen in different places, a model accounting such difference will make each event interact with different nodes. In the context of complex CPSs, nodes can be instantiated to represent a physical separation of processes, or a logical separation for processes that are based on distinct sets of information. Nonetheless, these two use cases of nodes feature common characteristics:

- they are responsible of data related to the location they represent, and
- they provide a set of typed labels to specify their possibility to exchange data with other nodes.

In the case of a node modeling a virtual location, that node is defined as a Chips *object*. An object definition simply gets described as a set of instructions comprising the declaration of *contextual variables* (using the keyword “ctx” as in line 3 of Figure 3) and the declaration of *channels* (by writing an instruction of two identifiers as in line 4 and 5). Imperative code instructions can also be used in such definitions to define default values of contextual values, but such

```

1 object lobbyRingElement
2 with{
3     ctx int nbPlayers;
4     OtherLobby left;
5     OtherLobby right
6 }

```

Figure 3: An object (virtual location) representing an instance of a lobby for a multiplayer server model

details go beyond the scope of the system level coordination mechanisms studied in this document.

Chips functions can be *embedded* in nodes. When it is the case, functions can refer to contextual variables as if they were defined in their own scope. To do so, we use the `ctx.<ctx_variable_name>` notation in the code sections of the components using it.

Channels are the Chips way to specify how different nodes are connectable together. They feature a *channel type* being an arbitrarily named type (excluding Chips keywords). During the evaluation of the system description section, these channel types allow to ensure the components connected together are compatible.

The definition of a node is orthogonal to the definition of a function, therefore, Chips provides the mean to define functions that are inherently attached to a node. Such concept is particularly relevant when designing components that emulate the physical behavior of a device. This is why such components are defined with the keyword *physical*. The *object* keyword allows to design a node as an abstraction for regrouping functions under a single component with shared contextual values. Such objects cannot exist on their own, they must be instantiated as embedded by physical nodes of a Chips model.

This embedding concept gives rise to a second kind of directed graph, adjacent to the *functional graph* of the previous section. We will call this graph the *mapping graph*. Its vertices can either be functions or nodes, and the edges represent the belonging of a function or a node to another node. Such graph has to be acyclic as an object cannot embed itself.

The graph formalism applied to the concept of embedding allows, just like for the functional graph, to perform early checking of properties of a system, but also to construct upon it the last specificity of Chips, *collective primitives*.

1.3 Collective primitives

With the current set of model elements described for Chips, it is possible to define two things:

- the evolution of signals among components of a system and

- the embedding relation of processes with the parts of system that contain them.

However, when designing large systems, the declination of the signals when they have to be spread among many component can become hard, in particular when accounting local information. The current tools available would require to either duplicate our functional blocks as many times as there are different configurations for receiving the data. The same problem is true when it comes to aggregating signals.

Our proposition with the Chips language is to let the programmer design a new kind of operator to describe the way data is modified by the context in which it is spread or collected. Hence the concepts of *spread* and *collect* collective function definitions.

They are comparable to the idea of *Streams* in the java language, except that, instead of processing the evolution of an accumulated value across logical time, they process the evolution of an accumulated value across logical space (i.e. the stream is updated when it reaches a new components). Such operation is sensitive to the architecture of the model formed, so it cares about the path data takes to go from one node to another. Since the interconnection between nodes is specified by plugging node *channels* together, collective functions also make use of the channels defined to specify data propagation.

A collective function definition is composed of:

- a direction keyword (*spread* or *collect*),
- a default accumulator definition,
- a support node reference,
- a stream update code section,
- a *target* output expression,
- a specification of updated accumulator for each channel of the referenced node.

The direction *spread* signify that information transmitted originates from a single component and gets propagated to many others. The direction *collect* signify the information originates from many components and get merged together to become the input of a single one. The default accumulator definition respects the same rules of the regular Chips function input parameters definition, except that each variable declared as a parameter **must** be provided with a default value.

To reference a support node, simply use the name of the node definition to use as support. This allows to compute when the stream update process is relevant to apply, thus respecting the *mapping graph* for the propagation of data.

The target output expression must be of the type of the input of the Chips function that will receive the propagated data. It is annotated with the @ symbol.

Updated accumulators must be specified for each channel that is defined for the type of node that the collective function references.

- For *spread* operations, all of the channel accumulators but the one that provided the input accumulator to the collective function will be used.
- For *collect* operations, only the channel accumulator that goes in the direction of the collecting component will be used.

Chips provides a *default* accumulator keyword to define the updated accumulator expression for all the channels that do not have a specification yet. In the context of collective functions, the set of instructions for the stream update code section is the same as the one from regular functions. However, the semantics of expressions in this context differs in two different ways:

- each datatype is enhanced with a new value (represented by the keyword *stop*)
- some new variables are available by default (the *input* and channel related and variables).

1.3.1 Semantics for *stop*

The *stop* value is to signify that a variable must not be processed anymore during the propagation. Any component receiving a *stop* signal during a spread operation will propagate the stop signal to warn other connected components that they must keep processing information without waiting for a “fresh” value. Any component receiving a *stop* signal during a collect operation will assume the collect operation must be completed using the default accumulator instead of a pre-computed one. The *stop* keyword can be used, during the implementation of the code section of the collective function, as a value of any primitive type. Any operation applied to a *stop* operand will result in the *stop* value.

The stop keyword can be used in any expression of the stream update code section and the updated accumulators expressions.

1.3.2 Defaultly available variables

input is the read-only variable representing the value provided by the Chips function that is the source of the data spread/collected. In the context of a *spread* collective function, it should appear in expressions of the default accumulator definition only. In the context of a *collect* collective function, it should appear in expressions of the stream update code section and/or the updated accumulators expressions. A collective function **must** feature at least one occurrence of the *input* variable.

channel related variables are a way to refer to accumulators when they are provided through a specific channel of the node. These variables are read-only and can be accessed with the notation

```

1 object ContainerObject
2 with{
3   ctx float myValue;
4   ctx bool participating;
5   upChannel up;
6   downChannel down;
7   leftChannel left;
8   rightChannel right;
9 }

1 collect (float accumulation = 0., int nbJumps = 0)
2 ponderatedSum among ContainerObject{
3   newAcc = up.accumulation * up.nbJumps
4           + down.accumulation * down.nbJumps
5           + left.accumulation * left.nbJumps
6           + right.accumulation * right.nbJumps;
7   int newNbJumps = 1;
8   if(newNbJumps < (1 + up.nbJumps)){newNbJumps = 1 + up.nbJumps; }
9   if(newNbJumps < (1 + down.nbJumps)){newNbJumps = 1 + down.nbJumps; }
10  if(newNbJumps < (1 + right.nbJumps)){newNbJumps = 1 + right.nbJumps; }
11  if(newNbJumps < (1 + left.nbJumps)){newNbJumps = 1 + left.nbJumps; }
12  if((newAcc == 0.) && !ctx.participating){
13    newAcc = stop;
14  }
15  }->@ (newAcc*(float)newNbJumps)
16  }->up(newAcc*1.1, newNbJumps)
17  }->default(newAcc, newNbJumps)

```

Figure 4: An object definition and a collective function to compute a sum of all the functions among the defined object, ponderated by the longest path to a leaf of the tree formed by the configuration used to collect data, putting an emphasis on data propagated via the *up* channel.

`<channel_name>.<accumulator_parameter_name>`

within any expression of the correct type. In case the data read hasn't been initialized because no component is connected through the given channel or because the data transmitted contained a *stop* value, the values of the parameters for that channel are the default values defined for the accumulator of this collective function definition.

Figure 4 presents an example of definition for a collective function featuring all the mentioned characteristics above. This example is pretty absurdly complicated for the sake of demonstrating all the usable features of collective function definitions. For this example to be fully functional, we assume that contextual values are previously set by other Chips functions linked to each instance of *ContainerObject*. We also assume the objects are correctly connected together by associating each channel of a given *ContainerObject* to at most one channel of another *ContainerObject*.

It is worth noting that for each channeled accumulator defined, the types of the provided expression respect the types of the default accumulator. If it hasn't been the case, an error would have been raised.

2 Formalisation of Chips modeling semantics

This section is dedicated to mathematically describe the elements of the Chips language that were described earlier in a less formal way. At first, we will define the formalism related the language *functional graphs* (graphs related to the functional block dependencies). Secondly, we will put our focus on the *mapping graphs* (graphs related to the association of functions and virtual objects to physical devices). Finally, we will see how the interconnections between virtual and physical objects define the structures of the datastreams that are profiled and aggregated among the system when using Chips collective primitives.

2.1 On functional graphs

The Chips language uses three kinds of primitive types being *integers*, *floating point values* and *booleans*. The sets of all possible values of these types will respectively be denoted *int*, *float* and *bool*, as in current general purpose programming languages. We assume their encoding being implementation dependant so we will not further describe these sets or specific restrictions applied to the operations applied to them.

Notation We denote the set of Chips types $\mathbb{T}$ as finite arrays of the primitive types *int*, *float* and *bool*:

$$\mathbb{T} = \{t^i \mid i \in \mathbb{N}^*, t \in \{int, float, bool\}\}$$

When mapped by identifiers, types constitute the base element for the construction of function parameter (whether input or output). Let S (for strings) be the set of all possible string sequences that are valid identifiers. From the Chips grammar (Appendix A), we see that this set is exactly the language generated from the regular expression `[a-zA-Z][a-zA-Z09_]*`, deprived of the set of the Chips keywords.

To assign values to parameters or variables, Chips defines instructions and expressions in a way that is similar to other imperative languages. For concision purposes, we will not detail their semantics and refer to [1] for examples of equivalent formal interpretation.

Notation For any $X : T^S$, a set of Chips variables, we will denote:

- $E[X]$ the set of correctly typed expressions making use of these variables,
- $value : T^{E[X]}$ the valuation function for a given expression over X into a value of type $T \in \mathbb{T}$ and
- $type : \mathbb{T}^{E[X]}$ the type deduction function for a correctly typed expression.

Definition (Inputs) Input parameters $\mathbb{I}$ for functions are tuples containing a name n , a type t , a value v and a boolean d to define if a default value is available for that parameter.

$$\mathbb{I} = \{(n, t, v, d) \mid n \in S, t \in \mathbb{T}, v \in t, d \in \{\top, \perp\}\} \quad (1)$$

Definition (Outputs) Output parameters $\mathbb{O}$ for functions are tuples containing a name n , a type t , an expression $e \in E[X_L]$ over a set of local variables $X_L : S \rightarrow T$ associated to the function.

$$\mathbb{O} = \{(n, e) \mid n \in S, e \in E[X_L]\} \quad (2)$$

To facilitate the expression of properties using inputs and outputs, we also define the following functions:

$$\begin{aligned} \text{name} : \begin{cases} (n, t, v, d) & \mapsto n \\ \mathbb{I} & \mapsto S \end{cases} & \quad \text{name} : \begin{cases} (n, e) & \mapsto n \\ \mathbb{O} & \mapsto S \end{cases} \\ \text{type} : \begin{cases} (n, t, v, d) & \mapsto t \\ \mathbb{I} & \mapsto \mathbb{T} \end{cases} & \quad \text{type} : \begin{cases} (n, e) & \mapsto \text{type}(e) \\ \mathbb{O} & \mapsto \mathbb{T} \end{cases} \\ \text{value} : \begin{cases} (n, t, v, d) & \mapsto v \\ \mathbb{I} & \mapsto T \end{cases} & \quad \text{value} : \begin{cases} (n, e) & \mapsto \text{value}(e) \\ \mathbb{O} & \mapsto T \end{cases} \\ \text{defaulted} : \begin{cases} (n, t, d) & \mapsto d \\ \mathbb{I} & \mapsto \{\top, \perp\} \end{cases} & \quad \text{value} : \begin{cases} (n, e) & \mapsto \text{value}(e) \\ \mathbb{O} & \mapsto T \end{cases} \end{aligned}$$

One obvious notion that has to be defined between objects like inputs and outputs is the *dataflow*.

Definition (Dataflow) A dataflow is a tuple $d = (o, i)$, where:

$$o \in \mathbb{O}, i \in \mathbb{I}, \text{type}(o) = \text{type}(i)$$

One can specify the behavior of functional blocks according to their inputs and outputs using a formalism close to the classic definition of a non-deterministic automaton, enhanced with local variables and a “readiness” property explained below. We will assume for the rest of this paper that the names of inputs, outputs and local variables do not overlap in the definition of the behavior of a function.

Definition (Function behavior) The behavior of a function with (evidently finite) inputs $I \subset \mathbb{I}$ and (also finite) outputs $O \subset \mathbb{O}$ is a tuple $B = (Q, \Sigma, \Delta, q_0, X_L, ready)$, where:

- Q is a set of states containing exactly $2^{|I|} + 2^{|O|}$ states:
 - 3 states representing the initial state q_{init} and the idle state $q_\emptyset$,
 - $2^{|I|} - 1$ states, each labelled by a different **non-empty** unordered set $\iota = \{name(i) \mid i \in I' \subseteq I\}$, which we will write q_ι for short,
 - $2^{|O|} - 1$ states, each labelled by a different **non-empty** unordered set $\omega = \{name(o) \mid o \in O' \subseteq O\}$, which we will write q_ω for short,
- $q_0 = q_{init}$;
- $ready \in \{\top, \perp\}$ arbitrarily set unless $\exists i \in I, \neg default(i)$. In that case, $ready = \perp$;
- X_L is a finite set of local data variables that associate an identifier in S to a value of type $T \in \mathbb{T}$;
- $\Sigma = \{name(x) \mid x \in I \cup O\} \cup \{\varepsilon\}$ serves as the alphabet of the automaton;
- Δ is the set of all the transition relations $\delta = (q, evt, \pi, g, q')$, such that:
 - $(q, q') \in \{(q_I, q_O), (q_{init}, q_I), (q_{init}, q_\emptyset)\} \cup \{(q_\iota, q_{\iota'}) \mid \iota \subset \iota' \subseteq \{name(i) \mid i \in I\} \wedge (|\iota| = |\iota'| - 1)\} \cup \{(q_\omega, q_{\omega'}) \mid \{\name(o) \mid o \in O\} \supseteq \omega \supset \omega' \wedge (|\omega'| = |\omega| - 1)\}$;
 - $evt \in \Sigma$ in such way that
 - $(q \in \{q_{init}, q_I\} \Leftrightarrow evt = \varepsilon)$
 - $\wedge (\forall \iota \subset I, \forall x \in I, (q, q') = (q_\iota, q_{\iota \cup \{name(x)\}}) \Rightarrow evt = name(x))$
 - $\wedge (\forall \omega \subset O, \forall x \in O, (q, q') = (q_{\omega \cup \{name(x)\}}, q_\omega) \Rightarrow evt = name(x));$
 - $\pi : I \times X_L \times O \mapsto I \times X_L \times O$ is the process to interpret whenever the transition is applied, ;
 - $g \in \{\top, \perp\}$ is a guard on the transition equal to $\perp$ iff $(ready \wedge (q = q_{init}) \wedge (q' = q_\emptyset)) \vee (\neg ready \wedge (q = q_{init}) \wedge (q' = q_I))$.

For any element $\delta = (q, evt, \pi, g, q')$ of Δ , we write $q(\delta)$, $evt(\delta)$, $\pi(\delta)$, $g(\delta)$ and $q'(\delta)$ as the respective components of δ .

Definition (Functional block) A Chips *functional block* is a tuple $f = (B, I, O, I_{default}, X_L, init, then, ready)$, where:

- $I \subset \mathbb{I}$ and $O \subset \mathbb{O}$ are finite;
- $I_{default} : \emptyset \mapsto I$ is the definition of default value for each input of F when provided. If a default value is not provided for an input parameter, then the *default* attribute of the input tuple is set to $\perp$, otherwise, it is set to $\top$;
- $X_L \in S \times T$ is a set of local variables;
- $init : \emptyset \mapsto S \times T$ is a process that defines the initial values of variables in X_L ;
- $then : I \times X_L \mapsto X_L \times O$ being the process that modifies the values of X_L and/or O according to the values of I and the past values of X_L .
- $B = (Q, \Sigma_f, \Delta, q_0, X_L, ready)$ is the function behavior with inputs I and outputs O , for which Δ is containing **exactly** the following transitions:

- $(q_{init}, \varepsilon, X_L := init(); I := I_{default}(), ready, q_I),$
- $(q_{init}, \varepsilon, X_L := init(); I := I_{default}(), \neg ready, q_\emptyset),$
- $(q_I, \varepsilon, (X_L, O) := then(I, X_L), \top, q_O),$
- Let D be the set of all the instanciated dataflows:
 $\forall \delta = (q, evt, \pi, g, q') \in \Delta,$
 $(\forall i \in I, (evt = name(i)) \wedge (\exists (o', i') \in D, name(i') = evt))$
 $\Rightarrow \pi \stackrel{\text{def}}{=} value(i) := value(o'))$ and
 $(\forall o \in O, (evt = name(o)) \wedge (\exists (o', i') \in D, name(o') = evt))$
 $\Rightarrow \pi \stackrel{\text{def}}{=} value(o') := value(o)).$

We will call $\mathbb{F}$ the set of possibly definable functional blocks. For this set, we define:

$$\begin{aligned}
 inputs : \begin{cases} (B, I, O, I_{default}, X_L, init, then, ready) & \mapsto I \\ \mathbb{F} & \mapsto \mathbb{I} \end{cases} \\
 outputs : \begin{cases} (B, I, O, I_{default}, X_L, init, then, ready) & \mapsto O \\ \mathbb{F} & \mapsto \mathbb{O} \end{cases}
 \end{aligned}$$

Definition (Operational semantics of Chips functional blocks) The operational semantics of a functional block $f = (B, I, O, I_{default}, X_L, init, then, ready)$ is given by the LTS $\sigma(f) = (Q \times X_L \times \mathbb{I}, \Sigma, \rightarrow)$ where:

- Q is the set of states of the functional block behavior B ,
- Σ is the alphabet of that same B ,

- $T \in \mathbb{T}$ and
- Let (q, x) be the control state of f with $x \in X_L$, $\rightarrow$ is the minimal transition relation inductively defined by the following rule:

$$\frac{q = q(\delta) \quad a = evt(\delta) \quad q' = q(\delta) \quad g(\delta) = \top \quad (inputs(f), x', outputs(f)) = \pi(\delta)(inputs(f), x, outputs(f))}{(q, x) \xrightarrow{a} (q', x')} \quad (3)$$

where $a \in \Sigma_f$ is any event that the function f can interpret, i.e. a writing of an input or a reading of an output.

A particular case of functional block is also worth describing because Chips allows to directly instantiate it with expressions instead of a definition of functional block plus a functional block instantiation:

$$CST : \begin{cases} \mapsto (B, I, O, I_{default}, X_L, init, then, ready) \\ e \mapsto (loop, \emptyset, \{("output", value(e))\}, \emptyset \mapsto \emptyset, \emptyset, 0_\Pi, 0_\Pi, \top) \\ E[X] \mapsto \mathbb{F} \end{cases} \quad (4)$$

Where *loop* is a 1-(initial)state, 1-transition, variable-free function behavior with no update process and an alphabet being the singleton $\{("output")\}$ and 0_Π is short for the empty process and X is a set of Chips variables.

Composition of functional blocks is achieved by connecting outputs to inputs. Thus defining *dataflows*.

Notation We write $D[F]$ the set of all the dataflows over a finite set of functional blocks $F \subset \mathbb{F}$.

Notation For any dataflow $d = (o, i)$, we note the source of d as $source(d) = o$ and the sink of d as $sink(d) = i$.

Definition (Functional graph) Functional graphs are, as described in Section 1.1, a directed graph $G_F = (F, D)$ for which vertices $F \in \mathbb{F}$ (for functions) are instances of functional blocks and edges $D \subset D[F]$ (for dataflows) are the input/output relations between such blocks. A functional graph G_F holds the following properties:

- (input completeness)

$$\forall f \in F, \forall i \in inputs(f), \exists d \in D, i = d_{sink} \quad (5)$$

- (output completeness)

$$\forall f \in F, \forall o \in outputs(f), \exists d \in D, o = d_{src} \quad (6)$$

- (dataflow sources unicity)

$$\forall o \in \text{outputs}(f), (\exists d1 \in D, o = d1_{src}) \wedge (\exists d2 \in D, o = d2_{src}) \Rightarrow d1 = d2 \quad (7)$$

- (dataflow sinks unicity)

$$\forall i \in \text{inputs}(f), (\exists d1 \in D, i = d1_{snk}) \wedge (\exists d2 \in D, i = d2_{snk}) \Rightarrow d1 = d2 \quad (8)$$

Such kind of graphs represents the composition of different functions achieving the functionalities of a designed system. To compute the behavior of such system, we describe hereafter the operational semantics of functional graphs.

Definition (Functional graphs operational semantics) Let $G_F = (F, D)$ a functional graph achieving input/output completeness and dataflow sources and sinks unicity. The operational semantics of G_F is given by

2.1.1 Properties of Chips functional graphs

In this (subsub)section, we must define:

- functional completeness (and talk about how it is achieved when compiling models?): all outputs are feeding inputs, all inputs are fed by outputs.
- dataflow sources and sinks (DSS) unicity: no two outputs feed the same input, no two inputs are fed by the same output.
- proof of deadlock freedom for complete graphs with DSS unicity ?
- feedback mechanisms (cyclic graphs)
- implicit logical clock cycles description

2.2 On mapping graphs

In this (sub)section, we must define:

- formalism definition for mapping graphs
- properties that must hold for correct Chips models

2.2.1 Definitions over mapping graphs

In this (subsub)section, we must define:

- node channels
- node contextual data

- node types: physical and objects
- linking (embedding) functions to nodes
- linking (embedding) objects to physicals
- plugging channels together

2.2.2 Properties of Chips mapping graphs

In this (subsub)section, we must define:

- configurations (embedding DAGs)
- hierarchical composition?
- functional graph mapping: check if a configuration can support a logical synchronous program
- envisioned properties for model checking (reconfigurability, timing robustness, imprecision robustness, others...?)

2.3 On collective primitives

In this (sub)section, we must define:

- formalism definition for spread and collect
- properties enabled by this concept

2.3.1 Formal definitions for collective primitives

In this (subsub)section, we must define:

- relation with functional graph (safe breaking of DSS unicity)
- relation with mapping graph (more particularly nodes and the concept of configuration)

2.3.2 Properties enabled by collective primitives

In this (subsub)section, we must define:

- implementation of control motifs
- sensitivity/robustness to reconfigurations

3 Current Chips implementation

Must talk about:

- compilation pipeline
- example implemented

4 Current, Future and Related Works

4.1 Current and future

Must talk about:

- custom data structures
- specification files for domain specific property checking
- example implemented for JavaBIP
- embeddable code generation
- reconfiguration specifications

4.2 Related

Maybe it should be placed somewhere else in the paper if it is a paper.

Must talk about:

- Lustre, Esterel, Signal, Fiacre, VHDL, Amaranth, Eclat, ScaFi, BIP, Tina, Hippo
- GenoM specifications for hierarchical models

5 Conclusion

On a vraiment fait du bon travail, c'est super!

References

- [1] No author. *No title*.

A Chips grammar

The current syntax of the language is defined hereafter in the style of ANTLR4 (BNF enhanced with regular expression elements).

```
grammar Chips;
program
    : preamble* system? EOF
    ;
system
    : SYSTEM_KW L_CURL
      s_statement*
      R_CURL
    ;
preamble
    : object_def
    | function_def
    | collective_op_def
    ;
object_def
    : OBJECT_KW IDENTIFIER with_section
    ;
node_mapping
    : HAVING_KW IDENTIFIER AS_KW IDENTIFIER SEMICOL
    ;
function_def
    : l_function_def
    | p_function_def
    ;
collective_op_def
    : c_signature L_CURL
      c_statement*
      R_CURL
      ARROW TARGET_KW L_PARENTH c_expr (COMMA c_expr)* R_PARENTH
      c_output+
    ;
c_output
    : ARROW DEFAULT_KW L_PARENTH c_expr (COMMA c_expr)* R_PARENTH
    | ARROW IDENTIFIER L_PARENTH c_expr (COMMA c_expr)* R_PARENTH
    ;
l_function_def
    : LOGICAL_KW IDENTIFIER L_PARENTH
      (df_parameter_decl (COMMA df_parameter_decl)*)? R_PARENTH
      init_section
      then_section
      named_output*
```



```

;
p_function_def
: PHYSICAL_KW IDENTIFIER L_PARENTH
  (pdf_parameter_decl (COMMA pdf_parameter_decl)*)? R_PARENTH
  with_section
  init_section
  then_section
  p_named_output*
;
c_signature
: c_keywords L_PARENTH
  (cdf_defaulted_decl (COMMA cdf_defaulted_decl)*)? R_PARENTH
  IDENTIFIER AMONG_KW IDENTIFIER
;
c_keywords
: SPREAD_KW
| COLLECT_KW
;
with_section
: WITH_KW L_CURL
  with_statement*
  R_CURL
;
with_statement
: IDENTIFIER IDENTIFIER SEMICOL
| CTX_KW df_type suffixes IDENTIFIER (ASSIGN expr)? SEMICOL
| statement
;
init_section
: INIT_KW L_CURL
  (statement)*
  R_CURL
;
then_section
: THEN_KW L_CURL
  (statement)*
  R_CURL
;
expr
: expr0 LT expr
| expr0 GT expr
| expr0 LEQ expr
| expr0 GEQ expr
| expr0 NEQ expr
| expr0 EQ expr
| expr0 AND expr

```

```

    | expr0 OR expr
    | expr0
    ;
expr0
: expr01 PLUS expr0
| expr01 MINUS expr0
| expr01
;
expr01
: MINUS expr1
| expr1
;
expr1
: expr2 TIMES expr1
| expr2 DIV expr1
| expr2 MOD expr1
| NOT expr2
| expr2
;
expr2
: INT
| FLOAT
| BOOL
| IDENTIFIER suffixes
| L_PARENTH expr R_PARENTH
| CTX_KW PERIOD IDENTIFIER suffixes
| IDENTIFIER L_PARENTH (expr (COMMA expr)*)? R_PARENTH
| cast
;
cast
: L_PARENTH df_type R_PARENTH expr
;
c_expr
: c_stopless_expr
| STOP_KW
;
c_stopless_expr
: c_stopless_expr0 LT c_stopless_expr
| c_stopless_expr0 GT c_stopless_expr
| c_stopless_expr0 LEQ c_stopless_expr
| c_stopless_expr0 GEQ c_stopless_expr
| c_stopless_expr0 NEQ c_stopless_expr
| c_stopless_expr0 EQ c_stopless_expr
| c_stopless_expr0 AND c_stopless_expr
| c_stopless_expr0 OR c_stopless_expr
| c_stopless_expr0

```

```

;
c_stopless_expr0
: c_stopless_expr01 PLUS c_stopless_expr0
| c_stopless_expr01 MINUS c_stopless_expr0
| c_stopless_expr01
;
c_stopless_expr01
: MINUS c_stopless_expr1
| c_stopless_expr1
;
c_stopless_expr1
: c_stopless_expr2 TIMES c_stopless_expr1
| c_stopless_expr2 DIV c_stopless_expr1
| c_stopless_expr2 MOD c_stopless_expr1
| NOT c_stopless_expr2
| c_stopless_expr2
;
c_stopless_expr2
: IDENTIFIER c_suffixes
| INT
| FLOAT
| BOOL
| INPUT_KW
| CTX_KW PERIOD IDENTIFIER c_suffixes
| IDENTIFIER PERIOD IDENTIFIER c_suffixes
| IDENTIFIER L_PARENTH (c_expr (COMMA c_expr)*)? R_PARENTH
| L_PARENTH c_stopless_expr R_PARENTH
| c_cast
;
c_cast
: L_PARENTH df_type R_PARENTH c_stopless_expr
;
suffixes
: (L_SQUA expr R_SQUA)*
;
c_suffixes
: (L_SQUA c_stopless_expr R_SQUA)*
;
s_suffixable_expr
: IDENTIFIER
| IDENTIFIER (L_PARENTH (expr (COMMA expr)*)? R_PARENTH)?
| block PERIOD IDENTIFIER
;
block
: IDENTIFIER suffixes
;

```

```

loop_in
    : IDENTIFIER (L_PARENTH (expr (COMMA expr)*)? R_PARENTH)?
    ;
loop_statement
    : FOREACH_KW IDENTIFIER IN_KW loop_in L_CURL
      statement*
      R_CURL
    ;
c_loop_statement
    : FOREACH_KW IDENTIFIER IN_KW loop_in L_CURL
      c_statement*
      R_CURL
    ;
s_loop_statement
    : FOREACH_KW IDENTIFIER IN_KW s_suffixable_expr L_CURL
      s_statement*
      R_CURL
    ;
if_else_statement
    : if_statement
      ELSE_KW L_CURL
      (statement)*
      R_CURL
    ;
s_if_else_statement
    : s_if_statement
      ELSE_KW L_CURL
      s_statement*
      R_CURL
    ;
c_if_else_statement
    : c_if_statement
      ELSE_KW L_CURL
      c_statement*
      R_CURL
    ;
if_statement
    : IF_KW L_PARENTH expr R_PARENTH L_CURL
      (statement)*
      R_CURL
    ;
s_if_statement
    : IF_KW L_PARENTH expr R_PARENTH L_CURL
      s_statement*
      R_CURL
    ;

```

```

c_if_statement
: IF_KW L_PARENTH c_expr R_PARENTH L_CURL
  c_statement*
  R_CURL
;
statement
: df_type suffixes IDENTIFIER (ASSIGN expr)? SEMICOL
| IDENTIFIER suffixes ASSIGN expr SEMICOL
| CTX_KW PERIOD IDENTIFIER suffixes ASSIGN expr SEMICOL
| loop_statement
| if_else_statement
| if_statement
;
s_statement
: IDENTIFIER suffixes IDENTIFIER SEMICOL
| block PERIOD IDENTIFIER L_PARENTH s_expr R_PARENTH SEMICOL
| LINK_KW IDENTIFIER suffixes TO_KW IDENTIFIER suffixes SEMICOL
| s_loop_statement
| s_if_else_statement
| s_if_statement
| statement
;
s_expr
: block PERIOD IDENTIFIER
| collective_operation block PERIOD IDENTIFIER
| expr
;
collective_operation
: L_PARENTH IDENTIFIER R_PARENTH
;
c_statement
: cdf_full_declaration SEMICOL
| IDENTIFIER c_suffixes ASSIGN c_expr SEMICOL
| CTX_KW PERIOD IDENTIFIER c_suffixes ASSIGN c_expr SEMICOL
| c_loop_statement
| c_if_else_statement
| c_if_statement
;
named_output
: ARROW IDENTIFIER L_PARENTH expr (COMMA expr)* R_PARENTH
;
p_named_output
: ARROW ACTUATOR_KW IDENTIFIER L_PARENTH expr (COMMA expr)* R_PARENTH
| named_output
;
df_parameter_decl

```

```

        : df_type suffixes IDENTIFIER (ASSIGN expr)?
    ;
df_type
    : INT_KW
    | FLOAT_KW
    | BOOL_KW
    ;
pdf_parameter_type
    : df_type suffixes
    | SENSOR_KW df_type suffixes
    ;
pdf_parameter_decl
    : pdf_parameter_type IDENTIFIER (ASSIGN expr)?
    ;
cdf_defaulted_decl
    : df_type suffixes IDENTIFIER ASSIGN c_expr
    ;
cdf_full_declaration
    : df_type suffixes IDENTIFIER (ASSIGN c_expr)?
    ;
SYSTEM_KW          : 'system'|'SYSTEM';
INT_KW              : 'int';
FLOAT_KW            : 'float';
BOOL_KW             : 'bool';
LOGICAL_KW          : 'logical';
PHYSICAL_KW         : 'physical';
AS_KW               : 'as';
INIT_KW             : 'init';
THEN_KW             : 'then';
FOREACH_KW          : 'for';
IN_KW               : 'in';
IF_KW               : 'if';
ELSE_KW             : 'else';
TO_KW               : 'to';
LINK_KW             : 'link';
HAVING_KW           : 'having';
INPUT_KW            : 'input';
STOP_KW             : 'stop';
AMONG_KW            : 'among';
SPREAD_KW           : 'spread';
COLLECT_KW          : 'collect';
CTX_KW              : 'ctx';
OBJECT_KW           : 'object';
WITH_KW             : 'with';
BY_KW               : 'by';
TARGET_KW           : '@';

```

```

DEFAULT_KW      : 'default';
USING_KW        : 'using';
ACTUATOR_KW     : 'actuator';
SENSOR_KW       : 'sensor';
ARROW           : '->';
PLUS            : '+';
MINUS           : '-';
TIMES           : '*';
DIV             : '/';
MOD             : '%';
LT              : '<';
GT              : '>';
EQ              : '==';
LEQ             : '<=';
GEQ             : '>=';
NEQ             : '!=';
AND             : '&&';
OR              : '||';
NOT             : '!';
ASSIGN          : '=';
COMMA           : ',';
SEMICOL         : ';';
L_PARENTH       : '(';
R_PARENTH       : ')';
L_CURL          : '{';
R_CURL          : '}';
L_SQUA          : '[';
R_SQUA          : ']';
COLUMN          : ':';
PERIOD          : '.';
BOOL            : 'true' | 'false';
fragment DIGIT   : [0-9] ;
fragment DOTTED_NUMBER
: '.'DIGIT+
  | DIGIT+'.'DIGIT*
;
fragment ID_START : [a-zA-Z];
fragment ID_NEXT  : [a-zA-Z0-9_];
FLOAT            : DOTTED_NUMBER;
INT              : DIGIT+;
IDENTIFIER        : ID_START ID_NEXT*;
NEWLINE          : ('\r'? '\n')+ -> skip ;
WS               : [ \t\r]+ -> skip ;
COMMENT          : '//' ~[\r\n]* -> skip ;

```